

AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

FACULTY OF SCIENCE & TECHNOLOGY

FALL 2025-26

COMPUTER GRAPHICS

Section: J (GROUP-A)

Project On:

Urban scenario with day-night transition in different seasons and weather.

Supervised By:

DIPTA JUSTIN GOMES

Submitted By:

Name	ID	Contribution
1.Md. Daud Ibrahim	**22-49760-3**	Scenario 1 design, keyboard control implementation, code review and debugging, code fixing, and Report writing.
2. Nafizur Rahman Iram	**23-52083-2**	Scenario 2 design, vehicle and object implementation, assistance in report writing, and testing support.
3. Yeasin Ahmed	**23-53014-3**	Scenario 3 design, object placement and alignment, visual testing.

1. **Introduction:**

This project presents an interactive computer graphics application developed using OpenGL and GLUT that demonstrates three visually distinct urban scenarios with real-time animation and user-controlled environmental effects. The project is meant to learn about the basics of computer graphics like making shapes, moving things around and making things happen at the time. It also shows how to use the keyboard to make things happen on the screen. Each city scene is different. It has things like cars moving, clouds floating, stars twinkling and weather like rain and snow that you can see happening in real time. The project uses things like timer functions to make the animation happen and keyboard input to let you control what is happening on the screen. The city scenes are like worlds, with moving vehicles and changing weather and you can play with them to see how they work. The system emphasizes clarity and modularity in design, allowing individual scene components and visual effects to be enabled or disabled independently, thereby supporting both educational understanding and practical experimentation with 2D graphics rendering techniques

Scenario Explanation:

Scenario 1: Scenario 1 is like a city street. You have an apartment building that is tan in color. Around it there are trees and benches where people can sit. There are also lamp posts and cars moving around. When it is daytime, the sky is bright. You can see the sun. There is also an air balloon that floats across the top of the screen. At night the sky gets darker. The sun is gone. Now you can see the moon. The windows of the apartment building are lit up. The streetlights are on. This makes the whole scene look more real. The city feels like it is really alive at night with all the lights on. Scenario 1 is about showing what a city street looks like during the day and at night. Winter mode can be activated where animated snow particles fall continuously and visually accumulate on tree canopies, benches, and selected ground-level objects. This combination of lighting changes, object animation, and weather effects demonstrates realistic environmental transitions within a 2D city setting

Scenario 2: shows us a neighborhood. This neighborhood has a house that's red and it has a chimney. There is also a house with green gables. Then there is a big grey building, in the middle. All of these are surrounded by trees that're always green.
When it is daytime, the sun shines brightly in the sky. You can see an airplane flying across the scene. When it becomes night, the sky gets dark. You can see the moon now. The green bus and the red truck turn on their headlights. An optional snowfall effect adds visual depth by rendering falling snowflakes over rooftops and trees, enhancing the sense of environmental change within the scene.

Scenario 3: Features a large symmetrical palace with three blue domes as the central structure of the scene. The environment includes surrounding buildings, trees, streetlamps, and a road with a

moving car. During daytime, the scene appears bright with a clear sky and visible sun. When the snowfall effect is enabled, snow particles fall across the palace, trees, and road, creating a winter atmosphere. This scenario demonstrates architectural modeling, object animation, and weather-based visual transitions in a 2D OpenGL environment.

Motivation:
This project is primarily intended for educational demonstration rather than real-world deployment. Contemporary computer graphics education necessitates comprehensive projects that transcend simple geometric rendering to demonstrate integrated understanding of transformation mathematics, rendering pipeline stages and input handling and visual optimization. This project addresses this requirement by synthesizing multiple graphics concepts into a cohesive and interactive application that students can analyze, modify, and extend. The realistic visualization of atmospheric phenomena particularly precipitation and accumulation effects represents a significant challenge in computer graphics.

Novelty:
The novelty of this project lies in the integration of multiple interactive features within a single 2D OpenGL application. The project combines three switchable urban scenarios with dynamic day–night transitions, animated vehicles, and weather effects such as rain and snowfall. Snow accumulation is visually represented on buildings, trees, and ground surfaces using geometric placement and conditional rendering. All elements are created procedurally using OpenGL primitives without relying on external assets, highlighting an efficient and educational approach to interactive urban scene visualization.

Background of the scenario:
The background of this work is rooted in the critical field of Urban Visualization, a domain essential for urban planners, architects and environmental scientists. By rendering realistic municipal environments in real-time, these technologies enable the preview of proposed structures within existing contexts and the analysis of traffic and pedestrian flows and the assessment of seasonal environmental impacts. This project serves as a functional prototype for such simulations, allowing stakeholders to visualize how weather effects, like toggleable snowfall, and lighting transitions affect the aesthetic and functional character of different urban zones. These scenarios—residential, industrial, and mixed-use—act as canonical models that demonstrate the broad applicability of rendering techniques in municipal planning and emergency response training.

Background of the platform used:
Technologically the application is operated by OpenGL and the GLUT utilizing the fixed-function pipeline to highlight the fundamental mathematical operations underlying modern graphics. Built in C++ and compiled via MinGW, the project prioritizes systems-level efficiency and direct GPU communication over heavy abstractions. This procedural architecture facilitates

a seamless Day/Night Transition System, where global color interpolation such as glowing windows and active streetlamps is managed through scenario isolated rendering functions. By utilizing professional-grade tools like GCC/MinGW, the development environment provides a robust platform for testing complex algorithms, ensuring that the final visualization remains portable, efficient, and educationally transparent.

2. Related Works

Computer graphics projects based on OpenGL and GLUT have been widely used in academic and practical applications to demonstrate real-time rendering, animation, and interaction techniques. These projects commonly focus on scene construction, object transformation, environmental effects, and user-controlled animation. The current project, which includes multiple animated 2D scenarios such as city environments, vehicles, weather effects, and day–night transitions, aligns closely with several existing OpenGL-based works. This section reviews ten relevant projects that are technically and conceptually related to the proposed system.

1. OpenGL 2D City Scene with Day–Night Transition
URL: https://www.openglprojects.in/2022/07/simple-city-scenery-opengl-computer.html
This project demonstrates a 2D city environment developed using OpenGL, featuring buildings, roads, vehicles, birds, and background elements. A key feature of the project is the implementation of a day–night transition using background color changes and conditional rendering of sun and moon objects. Animated elements such as moving vehicles and flying objects enhance scene realism. The overall structure and use of basic geometric primitives closely relate to the multi-scenario urban visualization approach adopted in the proposed project, particularly in terms of environmental transitions and animated scene components.

2. OpenGL Leeds Scene
URL: https://chaerim-kim.github.io/projects/OpenGL-leeds/
The OpenGL Leeds Scene is an interactive graphics project that visualizes famous landmarks in Leeds, including Millennium Square and university statues. The project incorporates camera controls, carousel animation, and user interaction through sliders. This work demonstrates scene composition and object animation techniques similar to the structured city environments used in the proposed project.

3. 3D City in OpenGL
URL: https://github.com/tars400/3Dcity-in-OpenGL
The 3D City in OpenGL project focuses on modeling a navigable urban environment with buildings, houses, and road networks. Users can explore the city through camera movement and perspective changes. Although this project is 3D-based, its architectural modeling approach and scene organization influenced the structural layout of buildings in the proposed 2D multi-scenario system.

4. 3D Car Animation in OpenGL
URL: https://github.com/saiminajagi/3D-car-animation-in-OpenGL
This project demonstrates animated car models with rotating wheels, adjustable camera angles, and interactive controls using keyboard and mouse inputs. Features such as background color changes, fog effects, and day–night driving modes are included. The vehicle motion logic and transformation techniques are directly relevant to the moving trucks, buses, and cars used in the proposed project.

5. Computer Graphics Mini Project: 3D Car
URL: https://www.openglprojects.in/2021/06/computer-graphics-mini-project-3d-car.html
This mini project presents a detailed car animation using OpenGL, focusing on object transformation, lighting, and smooth movement. The project emphasizes real-time animation and user interaction, which parallels the vehicle animation mechanisms implemented in the current system.

6. OpenGL Computer Graphics Animated Interactive Scene
URL: https://github.com/Martin777777/OpenGL-Computer-Graphics-Animated-Interactive-Scence
This project includes a rich animated scene developed using OpenGL and LWJGL. It integrates multiple moving objects, environmental elements, and interactive controls. The project demonstrates advanced animation sequencing and scene management, concepts that are also applied in organizing multiple scenarios within the proposed project.

7. OpenGL Particle Rain, Snow, and Hail Effects
URL: https://gist.github.com/4d9531cc9a7d1c34b998
This work focuses on implementing weather effects such as rain, snow, and hail using particle systems in OpenGL. Randomized particle positions and continuous motion create realistic precipitation effects. These techniques directly inspired the rain and snow animation logic used in the current project's weather simulation features.

8. Day–Night Cycle Animation Using OpenGL
URL: https://www.youtube.com/watch?v=4hd2--cVOBI
This project demonstrates a day–night cycle in a village environment, featuring sun and moon transitions, color shifts, and moving objects such as ships. The work highlights the use of time-based animation and conditional rendering, which closely aligns with the day–night toggle and lighting effects implemented in the proposed project.

9. OpenGL Town Project
URL: https://github.com/tavisit/OpenGL-Project
The OpenGL Town Project visualizes a small-town environment with buildings, street lights, statues, walls, and public spaces. The project emphasizes spatial organization and decorative

elements. Its approach to town layout and environmental detailing influenced the design of structured urban scenes in the proposed multi-scenario project.

APA References

Kim, C. (2020). *OpenGL Leeds scene*. https://chaerim-kim.github.io/projects/OpenGL-leeds/

Martin777777. (n.d.). *OpenGL computer graphics animated interactive scene* [GitHub repository]. https://github.com/Martin777777/OpenGL-Computer-Graphics-Animated-Interactive-Scence

OpenGL Projects. (2021). *Computer graphics mini project: 3D car*. https://www.openglprojects.in/2021/06/computer-graphics-mini-project-3d-car.html

OpenGL Projects. (2022). *Simple city scenery using OpenGL*. https://www.openglprojects.in/2022/07/simple-city-scenery-opengl-computer.html

Saimin Ajagi. (n.d.). *3D car animation in OpenGL* [GitHub repository]. https://github.com/saiminajagi/3D-car-animation-in-OpenGL

Tars400. (n.d.). *3D city in OpenGL* [GitHub repository]. https://github.com/tars400/3Dcity-in-OpenGL

Tavisit. (n.d.). *OpenGL town project* [GitHub repository]. https://github.com/tavisit/OpenGL-Project

YouTube. (n.d.). *Day–night cycle animation using OpenGL*. https://www.youtube.com/watch?v=4hd2--cVOBI

Anonymous. (2018). *OpenGL particle rain, snow and hail effects* [Gist]. https://gist.github.com/4d9531cc9a7d1c34b998

3. Implementation:

1.Key Features of the Project:

Multi-Scenario Interactive Visualization:
The application features three completely distinct scenarios, each with unique geometry, lighting characteristics, and environmental properties. Users can seamlessly switch between scenarios using keyboard controls (keys 1, 2, 3), with the rendering system dynamically loading and rendering the appropriate scene graph for each environment. This architecture demonstrates principles of scene management, where different logical scenes share a common rendering framework.

Advanced Snow Rendering System:
The snow rendering system is implemented as a dynamic environmental layer using particle-based animation and conditional rendering. Snow particles are rendered at varying positions and sizes to create visual depth, while accumulation effects are simulated geometrically on selected objects.

Dynamic Day-Night Cycle:
The system implements a continuous day-night cycle controlled via keyboard (keys 9/0 for manual toggle, or automatic progression). The implementation includes:

Time-Based Lighting: Ambient and diffuse lighting values transition smoothly between day (bright, whitish) and night (dark, bluish) configurations.

Background Color Transitions: Clear color values change from bright blue (day) to dark navy (night).

Object-Specific Adaptations: Building windows and lights can be toggled independently, and lamp posts render with illumination during night mode.

Complex Geometric Modeling:
Each scenario contains dozens of geometric primitives including:

Buildings: Multi-component structures with walls, roofs, doors, windows
Vegetation: Tree structures with foliage representation
Infrastructure: Lamp posts, benches, street elements
Vehicles: Dynamic vehicles that can move and interact with environments

Real-Time Interactive Control:
The application provides comprehensive keyboard controls for:

SCENARIO	KEY / INPUT	ACTION
GLOBAL	1 / 2 / 3	Switch Scenario 1 / 2 / 3
GLOBAL	9 / 0	Day / Night mode
SCENARIO 1	↑ / ↓	Increase / decrease truck speed
	← / →	Stop / reset truck
	A / B	Bus speed increase / decrease
	C / D	Reset / stop bus
	R / S	Enable / disable rain
	W / E	Enable / disable snow particles
	X	Toggle winter mode
SCENARIO 2	W / S	Bus speed increase / decrease
	R / Space	Reset / stop bus
	↑ / ↓	SUV speed control
	← / →	Stop / reset SUV
	Shift+W / Shift+S	Enable / disable snow
	X	Toggle winter mode
SCENARIO 3	Q / W	Stop / start car 1
	Mouse L / R	Increase / decrease car 1 speed
	E / R	Stop / start car 2
	T / Y	Increase / decrease car 2 speed
	U / I / O / P	Cloud movement
	M / B	Stop / start airplane
	L / K	Lights ON / OFF
	Shift+W / Shift+S	Enable / disable snow
	X	Toggle winter mode
	D / N	Day / Night mode

2. The key Technologies That We Use:

C++ Programming Language:
The project uses C++ because it is good at managing objects and controlling memory. This makes it easy to work with OpenGL. C++ is also very flexible which means we can use it to do lots of things like make objects move and change and create animations that look cool. The C++ language is great for making objects behave in a way, which is important, for our project. We like C++ because it helps us make things that work together.

OpenGL (Open Graphics Library):
OpenGL is what we use to make pictures on the screen. We need it to draw things like lines and circles and to make them move. It helps us draw lots of things like trees and buildings and rivers. We can even make clouds and stars with it. When we are making these things, we use functions like glBegin and glVertex2f. Glcolor3f a lot. OpenGL is really important, for making all these things look good on the screen.

GLUT (OpenGL Utility Toolkit):
The GLUT library makes it easy to create windows and handle events. It has functions such as glutDisplayFunc and glutIdleFunc and glutTimerFunc that help update the positions of objects and redraw the scene all the time. The GLUT library also takes care of keyboard input and mouse input for things like switching between scenarios and making things interactive, with GLUT.

Mathematical Transformations:
The sun and moon move in a circle. We use things like sine and cosine to figure out where they are. These are called functions. We also want things, like clouds and birds to look like they are really moving. So, we use some tricks like translation, scaling and rotation to make that happen.

Animation Logic:
The program uses variables to keep track of where objects are, how fast they are going and which way they are heading. We have flags that say what is happening right now like if it is day or night if the rain is, on or off and what scenario we are using. These variables get updated a bit at a time by timer functions so everything moves smoothly and looks good.

Color Interpolation:
The objects change color slowly. This happens because the colors mix together in a line. That means you do not see jumps, in color. It looks more real when the day turns into night and the sky and objects change color. The day-night changes look very natural because of this technique. The sky and object colors are what make it look so real.

Object Layering Techniques:
Things are drawn in an order: the background, the mid-ground and the foreground. This way when things overlap it looks like they really are in front of or behind each other, which helps us see how

deep things are even though we are looking at a picture with objects, like the background and the foreground.

List of Key Functions Used in the Project

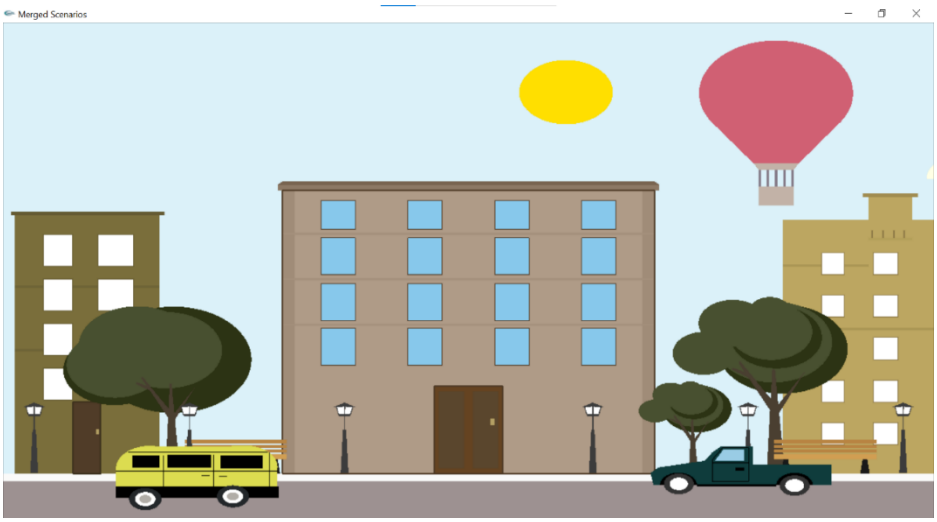
Core Rendering Function

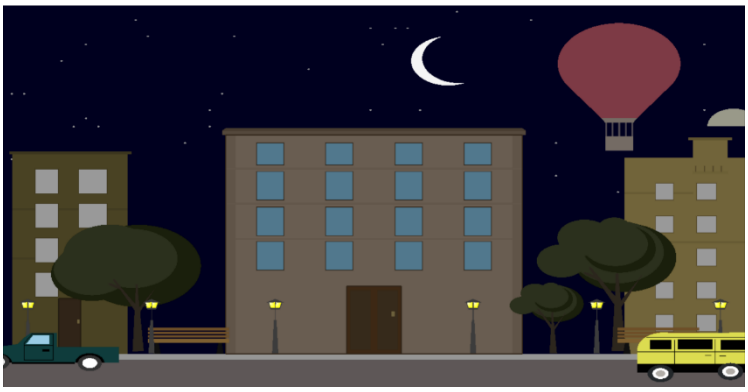
Category	Function Name	Purpose
Core Rendering	display()	Main rendering loop; draws active scenario, objects, weather, and lighting
Basic Shapes	drawCircle()	Draws circular objects (sun, moon, trees, snow, stars)
	drawSemiCircleFromPoints()	Draws semicircles for clouds, windows, and curves
Weather Effects	drawRain()	Renders falling rain
	updateRain()	Updates rain particle motion
	drawSnow()	Draws falling snow particles
	updateSnow()	Updates snow movement with drift
Snow Accumulation	drawSnowOnTrees()	Adds snow on trees
	drawSnowOnGround()	Draws snow on ground
	drawSnowOnBenchesAndLampsScenario1()	Snow on benches & lamps (Scenario 1)
	drawSnowOnTreesScenario2()	Snow on trees (Scenario 2)
	drawSnowOnRoofsScenario2()	Snow on roofs (Scenario 2)
	drawSnowOnLampPostsScenario2()	Snow on lamp posts (Scenario 2)
	drawSnowOnRoofsScenario3()	Snow on palace roofs (Scenario 3)

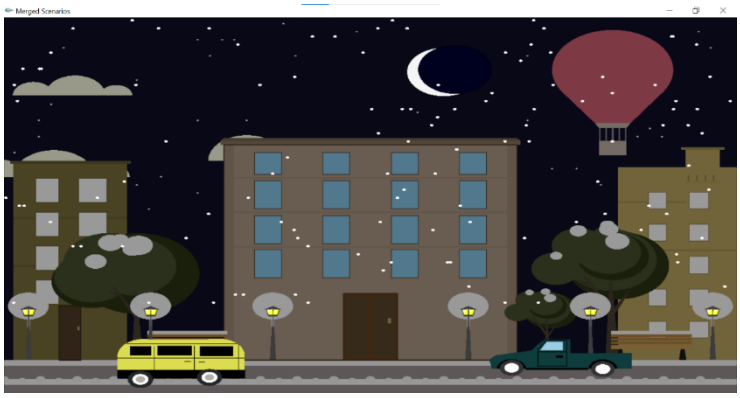
	drawSnowOnTreesScenario3()	Snow on trees (Scenario 3)
	drawSnowOnLampPostsScenario3()	Snow on lamp posts (Scenario 3)
Animation / Motion	updateTruck()	Animates truck (Scenario 1)
	updateBus()	Animates bus movement
	updateBalloon()	Floating motion of hot air balloon
	updateCloud()	Moves clouds across the sky
Sky Elements	drawSun()	Draws sun in day mode
	drawMoon()	Draws moon in night mode
	drawStars()	Draws stars at night
	drawCloud()	Draws cloud shapes
Buildings	house1()	Draws residential building
	house2()	Draws second residential building
	palace()	Draws palace structure
Windows & Doors	window1()	Draws arched window
	window2()	Draws rectangular window
	door()	Draws main entrance door
Vegetation	tree1()	Draws large tree
	tree2()	Draws secondary tree
	tree2_translated()	Reuses tree with scaling/translation
Decorative Objects	Airbln()	Draws and animates hot air balloon

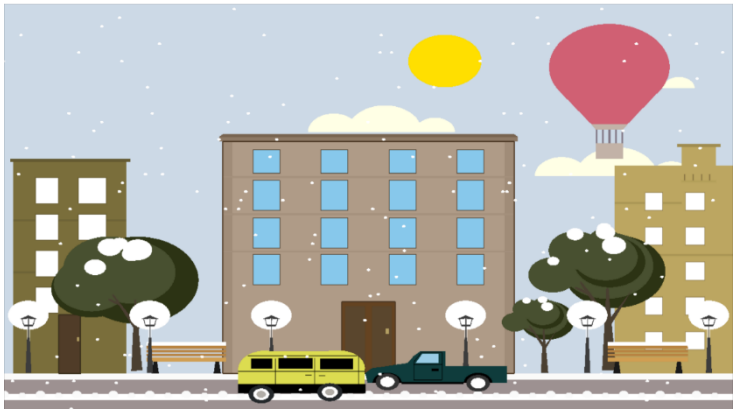
4. Screenshots of the project:

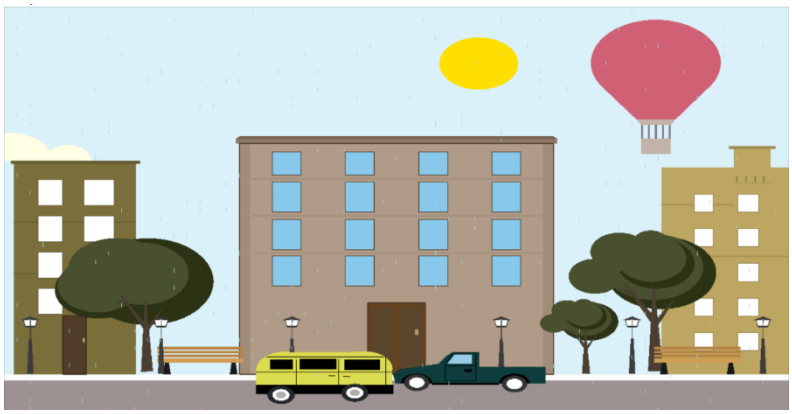
Scenario 1 Screenshots:



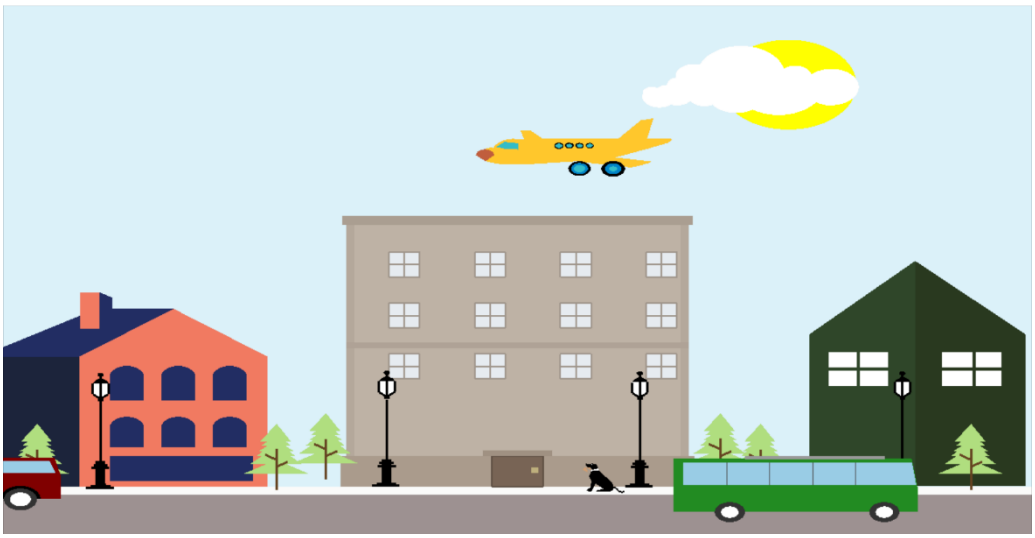






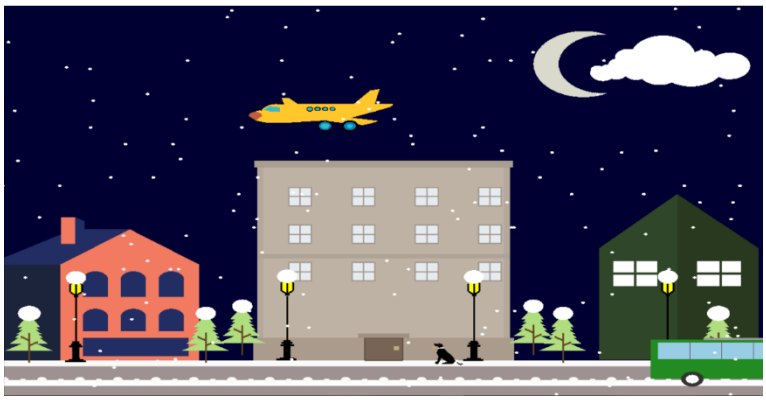


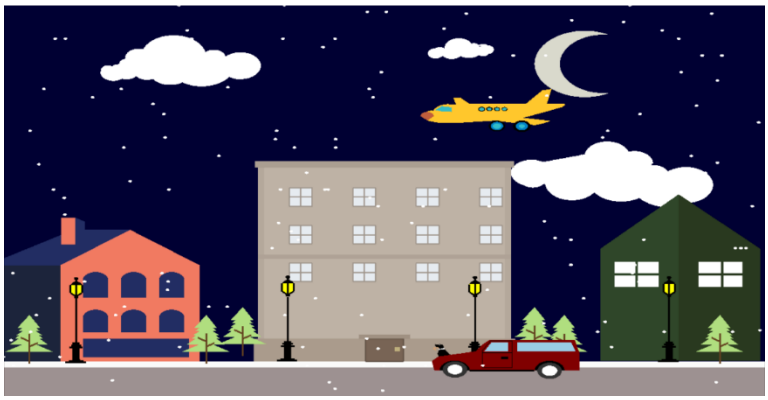
Scenario 2 Screenshots:



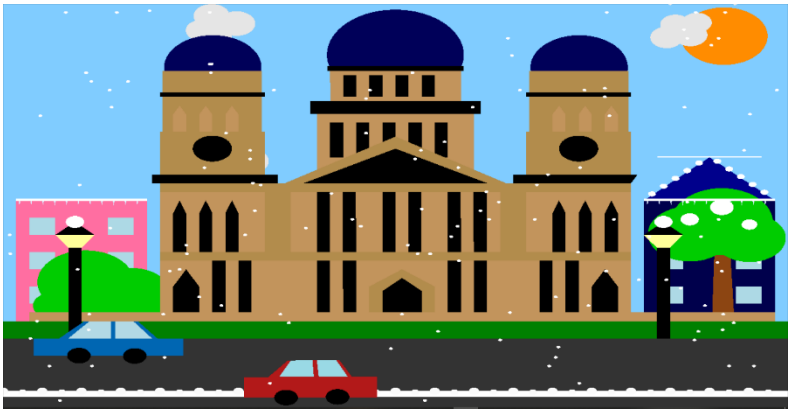






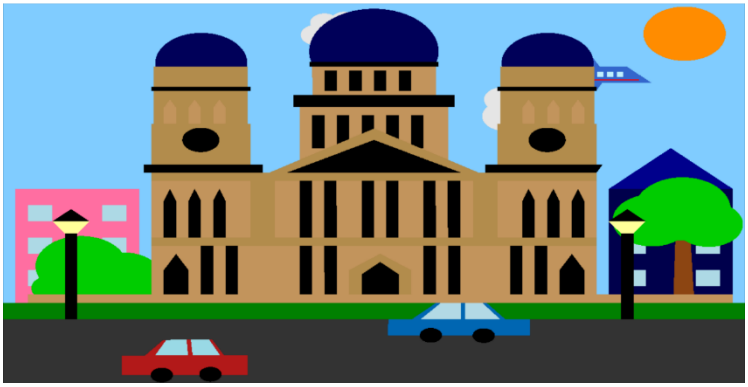


Scenario 3 Screenshots:











5. **Code availability:**

GITHUB repository→ https://github.com/MdDaudIbrahim/COMPUTER-GRAPHICS

Code Direct Link→ https://github.com/MdDaudIbrahim/COMPUTER-GRAPHICS/blob/main/GraphicsProject/main.cpp

6. Future Work:

1. Shader-based rendering (GLSL): Improve lighting realism and visual effects.
2. Physically based snow rendering: Add realistic accumulation and interaction.
3. Advanced weather system: Include fog, wind, and seasonal variations.
4. Improved geometric modeling: Enhance object detail and terrain features.
5. Physics engine integration: Enable realistic collisions and interactions.
6. Advanced lighting: Implement dynamic shadows and light attenuation.
7. Performance optimization: Improve rendering efficiency and particle handling.
8. Extended scenario support: Add more urban and environmental scenes.

7. Conclusion:

Project Limitations:

The project is pretty good at showing off computer graphics ideas, but it has some problems. The project has these problems because of how it was set up and how it was made. The program was made using a version of OpenGL, which is called the fixed-function pipeline. This old version does not let the project use computer graphics effects like special lighting, shadows and materials. All the objects in the project are made using shapes and basic building blocks which means they do not look very detailed. The project also cannot use models from outside the project, which's a limitation of the computer graphics concepts used in the project.

The project does not include a physics engine; therefore, object movement follows predefined paths and weather effects such as snow and rain are visually simulated without physical interaction or collision detection. Rendering is performed using immediate-mode functions (glBegin and glEnd), which are less efficient than modern rendering techniques and may impact performance as scene complexity increases. Additionally, the project is limited to a 2D environment and does not support camera navigation or true depth perception.

Brief Summary:

The project does what it is supposed to do in terms of education. It shows a 2D city model that you can play with. This is done using OpenGL and GLUT. The project has three scenes. Each scene has moving things in it like day turning into night and weather like rain and snow. The project uses math and other techniques to make these scenes work. It also lets you control what happens with the keyboard. The project uses modeling and timer-based animation to make everything look real. The project is good at demonstrating a 2D urban visualization system.

Overall, this work provides a clear and well-structured example of real-time graphics programming suitable for academic learning. The modular design and readable code structure make it a strong foundation for future improvements, including shader-based rendering, enhanced weather simulation, and more advanced interaction techniques.